

SIMATS ENGINEERING

CO3- ASSESSMENT TOOL 2

SIMULATION TASK

COURSE NAME: DIGITAL SIGNAL PROCESSING

COURSE CODE: ECA09

COURSE OUTCOME COVERED

CO 3: Evaluate finite word-length effects and multirate signal processing techniques, including decimation, interpolation, and polyphase decomposition. **Bloom Level mapped-BL3,BL4,BL5**

Topics Covered:

Quantization- Truncation and Rounding errors - Quantization noise - Limit cycle oscillations – Signal scaling. Parametric and Non parametric Spectral estimation. Decimation- Interpolation- Sampling rate conversion- Implementation of sampling rate conversion- Polyphase Decomposition - 5G Decimation/Interpolation: For mmWave beamforming.

Assessment Tool number	: CO3-AT2
Assessment Tool Weightage	: 35%
Assessment Tool Name	: Simulation Task
Duration of this assessment	: 60 Minutes
Marks	: 25 Marks
Type of Assessment Conduction	: Self Learning (Home Task)

- Each student will be assigned one simulation-based problem related to the course topics.
- Solve the assigned simulation problem independently.
- Develop and run the simulation using MATLAB/Simulink.
- Analyze and present the results with graphs and screenshots.
- Upload the simulation code and output to GitHub.
- Submit the GitHub repository link in Google Classroom (GCR).

Dr.

39. End-to-End 5G Multirate Receiver Simulation

Simulate an end-to-end 5G receiver employing multirate DSP techniques and evaluate signal quality. (25) (BL5) (WK4)

Given Data:

- Input Sampling Rate = 245.76 MHz
- Decimation Factor = 8
- OFDM Signal Bandwidth = 100 MHz

Tasks:

1. Generate the received signal.
2. Implement filtering.
3. Perform decimation.
4. Estimate output spectrum.
5. Evaluate signal quality.
6. Analyze processing efficiency.

Code of Conduct:

I YOGESWARAN K/192412615 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Simulation tools					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Title

End-to-End 5G Multirate Receiver Simulation Using Filtering, Decimation and Spectral Estimation

Aim

To simulate an end-to-end 5G-like OFDM receiver using MATLAB and evaluate the effect of **anti-aliasing filtering, decimation, spectral estimation, signal quality and processing efficiency**.

The given system uses an input sampling rate of **245.76 MHz**, a decimation factor of **8**, and an OFDM signal bandwidth of **100 MHz**.

Objectives

1. To generate a received 5G-like OFDM signal.
2. To introduce channel noise into the received signal.
3. To design and apply an anti-aliasing FIR filter.
4. To perform decimation by a factor of 8.
5. To estimate the signal spectrum using Welch's method.
6. To evaluate signal **End-to-End 5G Multirate Receiver Simulation** quality using SNR and EVM.
7. To compare the input and output spectra.
8. To analyze the reduction in sampling rate and processing requirements.
9. To study the bandwidth limitation caused by decimation.
10. To demonstrate the importance of anti-aliasing filtering in multirate DSP.

THEORY

A 5G receiver uses digital signal processing techniques to recover and process high-bandwidth communication signals efficiently. In this simulation, an OFDM-based received signal is considered with an input sampling rate of 245.76 MHz and an occupied bandwidth of 100 MHz. Orthogonal Frequency Division Multiplexing (OFDM) divides the available bandwidth into multiple closely spaced orthogonal subcarriers, making it suitable for high-speed 5G communication systems. QPSK modulation is used to generate complex-valued OFDM symbols, and a cyclic prefix is added to reduce the effect of inter-symbol interference. Noise is also introduced to represent the practical channel conditions experienced by a received communication signal.

Multirate signal processing is used to reduce the sampling rate while maintaining the required signal information. The given receiver performs decimation by a factor of 8, reducing the sampling rate from 245.76 MHz to 30.72 MHz. However, decimation must be performed carefully because the output Nyquist frequency becomes only 15.36 MHz. Since the original OFDM bandwidth is 100 MHz, the complete bandwidth cannot be retained at the lower sampling rate. Therefore, an anti-aliasing low-pass FIR filter is applied before decimation to

suppress frequency components that cannot be represented at the reduced sampling rate. This filtering stage prevents aliasing, which would otherwise cause unwanted spectral components to overlap with the desired signal.

The FIR filter is designed using a Hamming-window-based low-pass filter in MATLAB. The filter allows the desired lower-frequency components to pass while attenuating components close to and above the new Nyquist limit. After filtering, the signal is decimated by selecting every eighth sample. This reduces the number of samples that must be processed by the subsequent receiver stages and significantly lowers the computational and memory requirements. Thus, the simulation demonstrates the fundamental trade-off in multirate DSP between processing efficiency and the amount of signal bandwidth that can be preserved.

Spectral estimation is then performed to evaluate the frequency-domain characteristics of the received signal. Welch's power spectral density method, implemented using MATLAB's `pwelch` function, is used as a non-parametric spectral estimation technique. The input spectrum, filtered spectrum, and output spectrum after decimation are compared to observe the effect of anti-aliasing filtering and sampling-rate conversion. The spectrum provides a visual representation of the signal bandwidth and shows how unwanted frequency components are removed before the decimation stage.

Signal quality is evaluated using parameters such as Signal-to-Noise Ratio (SNR), Error Vector Magnitude (EVM), and RMS signal level. SNR indicates the ratio between the desired signal power and the unwanted error or noise power, while EVM measures the deviation of the received signal from the ideal reference signal and is particularly important for digitally modulated communication systems such as OFDM. These measurements allow the performance of the receiver before and after multirate processing to be compared quantitatively. A suitable anti-aliasing filter helps preserve the required portion of the signal while minimizing distortion caused by sampling-rate conversion.

The overall MATLAB model therefore represents an end-to-end multirate 5G receiver consisting of OFDM signal generation, noise addition, anti-aliasing FIR filtering, decimation, spectral estimation, and signal-quality analysis. The sampling rate is reduced by a factor of 8, resulting in an 87.5% reduction in the number of samples processed downstream. The simulation demonstrates that although aggressive decimation improves processing efficiency, it also imposes a strict bandwidth limitation. Hence, proper filtering and sampling-rate selection are essential in practical 5G receivers to achieve a balance between computational efficiency, spectral integrity, and communication signal quality.

PROGRAM

clc;

clear;

close all;

%%

=====

====

% END-TO-END 5G MULTIRATE RECEIVER SIMULATION

% CO3 - AT2

% Digital Signal Processing

%

=====

=====

%% 1. SYSTEM PARAMETERS

Fs_in = 245.76e6; % Input sampling rate

D = 8; % Decimation factor

BW = 100e6; % OFDM bandwidth

Fs_out = Fs_in / D; % Output sampling rate

Fnyq_in = Fs_in / 2; % Input Nyquist frequency

Fnyq_out = Fs_out / 2; % Output Nyquist frequency

fprintf('\n=====\\n');

fprintf(' END-TO-END 5G MULTIRATE RECEIVER\\n');

fprintf('=====\\n');

fprintf('Input Sampling Rate : %.2f MHz\\n', Fs_in/1e6);

fprintf('OFDM Bandwidth : %.2f MHz\\n', BW/1e6);

fprintf('Decimation Factor : %d\\n', D);

fprintf('Output Sampling Rate : %.2f MHz\\n', Fs_out/1e6);

fprintf('Output Nyquist Frequency : %.2f MHz\\n', Fnyq_out/1e6);

```
%%
```

```
=====
```

```
=====
```

```
% 2. GENERATE 5G-LIKE OFDM SIGNAL
```

```
%
```

```
=====
```

```
=====
```

```
Nfft = 8192;          % FFT size
```

```
Ncp = 512;           % Cyclic prefix
```

```
numSymbols = 20;      % Number of OFDM symbols
```

```
% 30 kHz subcarrier spacing
```

```
SCS = 30e3;
```

```
% Number of active subcarriers
```

```
% Approximately 100 MHz occupied bandwidth
```

```
numActive = 3333;
```

```
% Generate random QPSK data
```

```
bitsI = randi([0 1], numActive*numSymbols, 1);
```

```
bitsQ = randi([0 1], numActive*numSymbols, 1);
```

```
qpskData = ((2*bitsI - 1) + ...  
            1j*(2*bitsQ - 1)) / sqrt(2);
```

```
qpskData = reshape(qpskData, ...  
                   numActive, ...  
                   numSymbols);
```

```
%% Generate OFDM symbols
```

```
txSignal = [];
```

```
halfActive = floor(numActive/2);
```

```

center = Nfft/2 + 1;

activeIndices = [ ...
    center-halfActive:center-1, ...
    center+1:center+(numActive-halfActive)];

for k = 1:numSymbols

    % Frequency domain symbol
    X = zeros(Nfft,1);

    X(activeIndices) = qpskData(:,k);

    % IFFT
    x_ifft = ifft(ifftshift(X));

    % Add cyclic prefix
    x_cp = [ ...
        x_ifft(end-Ncp+1:end);
        x_ifft
    ];

    % Append symbol
    txSignal = [txSignal; x_cp];

end

%% Normalize signal

txSignal = txSignal / rms(txSignal);

fprintf('\nOFDM Signal Generated Successfully.\n');
fprintf('Number of OFDM Symbols : %d\n', numSymbols);
fprintf('FFT Size          : %d\n', Nfft);
fprintf('Cyclic Prefix       : %d\n', Ncp);

```

```
%%
```

```
=====
```

```
=====
```

```
% 3. ADD AWGN CHANNEL
```

```
%
```

```
=====
```

```
=====
```

```
SNR_input_dB = 25;
```

```
signalPower = mean(abs(txSignal).^2);
```

```
noisePower = ...
```

```
    signalPower / (10^(SNR_input_dB/10));
```

```
noise = sqrt(noisePower/2) * ...
```

```
    (randn(size(txSignal)) + ...
```

```
    1j*randn(size(txSignal)));
```

```
rxSignal = txSignal + noise;
```

```
fprintf('Input SNR          : %.2f dB\n', SNR_input_dB);
```

```
%% Check signal validity
```

```
if any(~isfinite(rxSignal))
```

```
    error('ERROR: Received signal contains NaN or Inf values.');
```

```
end
```

```
%%
```

```
=====
```

```
=====
```

```
% 4. INPUT SPECTRUM
```

```
%
```

```
=====
```

```
=====
```

```
Nspec = 65536;
```

```
[PSD_input, f_input] = pwelch( ...  
    rxSignal, ...  
    hamming(4096), ...  
    2048, ...  
    Nspec, ...  
    Fs_in, ...  
    'centered');
```

```
%%%
```

```
=====
```

```
====
```

```
% 5. ANTI-ALIASING FIR FILTER
```

```
%
```

```
=====
```

```
=====
```

```
% After decimation:
```

```
%
```

```
% Fs_out = 30.72 MHz
```

```
% Nyquist = 15.36 MHz
```

```
%
```

```
% Therefore, frequencies above approximately
```

```
% 15 MHz must be strongly attenuated.
```

```
Fpass = 12e6;          % Passband
```

```
Fstop = 15e6;          % Stopband
```

```
%% FIR filter design using fir1
```

```
filterOrder = 128;
```

```
% Normalized cutoff frequency
```

```
Wn = Fpass / Fnyq_in;
```

```
% Hamming-window FIR low-pass filter
```

```

b = fir1(filterOrder, Wn, ...
    'low', hamming(filterOrder+1));

a = 1;

%% Verify filter coefficients

if any(~isfinite(b))
    error('ERROR: FIR filter coefficients contain NaN or Inf.');
```

end

```

fprintf('\nAnti-Aliasing Filter:\n');
fprintf('Filter Type      : FIR Low-Pass\n');
fprintf('Filter Order       : %d\n', filterOrder);
fprintf('Passband            : %.2f MHz\n', Fpass/1e6);
fprintf('Approx. Cutoff      : %.2f MHz\n', Fpass/1e6);
fprintf('Output Nyquist      : %.2f MHz\n', Fnyq_out/1e6);

%%
=====

===

% 6. FILTER RECEIVED SIGNAL
%
=====

=====

filteredSignal = filtfilt(b, a, rxSignal);

%% Check filtered signal

if any(~isfinite(filteredSignal))
    error('ERROR: Filtered signal contains NaN or Inf values.');
```

end

```

fprintf('Anti-aliasing filtering completed successfully.\n');
```

```
%%
```

```
=====
```

```
=====
```

```
% 7. DECIMATION
```

```
%
```

```
=====
```

```
=====
```

```
rxDecimated = filteredSignal(1:D:end);
```

```
%% Check decimated signal
```

```
if any(~isfinite(rxDecimated))
```

```
    error('ERROR: Decimated signal contains NaN or Inf values.');
```

```
end
```

```
fprintf('\nDecimation completed successfully.\n');
```

```
fprintf('Input Samples      : %d\n', length(rxSignal));
```

```
fprintf('Output Samples       : %d\n', length(rxDecimated));
```

```
%%
```

```
=====
```

```
=====
```

```
% 8. OUTPUT SPECTRUM
```

```
%
```

```
=====
```

```
=====
```

```
[PSD_output, f_output] = pwelch( ...
```

```
    rxDecimated, ...
```

```
    hamming(1024), ...
```

```
    512, ...
```

```
    32768, ...
```

```
    Fs_out, ...
```

```
    'centered');
```

```
%%
```

```
=====
```

```
=====
```

```
% 9. FILTERED SPECTRUM
```

```
%
```

```
=====
```

```
=====
```

```
[PSD_filtered, f_filtered] = pwelch( ...  
    filteredSignal, ...  
    hamming(4096), ...  
    2048, ...  
    Nspec, ...  
    Fs_in, ...  
    'centered');
```

```
%%
```

```
=====
```

```
=====
```

```
% 10. SIGNAL QUALITY ANALYSIS
```

```
%
```

```
=====
```

```
=====
```

```
% Create matching reference signal
```

```
txReference = txSignal(1:D:end);
```

```
% Make sure both vectors have equal length
```

```
minLength = min(length(txReference), ...  
    length(rxDecimated));
```

```
txReference = txReference(1:minLength);
```

```
rxTest = rxDecimated(1:minLength);
```

```
%% Error signal
```

```
errorSignal = txReference - rxTest;
```

```
%% Output SNR
```

```
SNR_output_dB = ...  
    10*log10( ...  
    mean(abs(txReference).^2) / ...  
    mean(abs(errorSignal).^2));
```

```
%% EVM
```

```
EVM_percent = ...  
    100 * ...  
    rms(errorSignal) / ...  
    rms(txReference);
```

```
EVM_dB = 20*log10(EVM_percent/100);
```

```
%% RMS values
```

```
inputRMS = rms(rxSignal);  
filteredRMS = rms(filteredSignal);  
outputRMS = rms(rxDecimated);
```

```
%%
```

```
=====
```

```
===  
% 11. PROCESSING EFFICIENCY
```

```
%
```

```
=====
```

```
===  
  
inputSamples = length(rxSignal);  
outputSamples = length(rxDecimated);
```

```
sampleReduction = ...  
    (1 - outputSamples/inputSamples) * 100;
```

```
samplingRateReduction = ...  
    (1 - Fs_out/Fs_in) * 100;
```

```
processingReductionFactor = ...  
    Fs_in/Fs_out;
```

```
%%
```

```
=====
```

```
====
```

```
% 12. DISPLAY FINAL RESULTS
```

```
%
```

```
=====
```

```
=====
```

```
fprintf('\n=====\\n');
```

```
fprintf('          FINAL RESULTS\\n');
```

```
fprintf('=====\\n');
```

```
fprintf('\nSAMPLING RATE:\\n');
```

```
fprintf('Input Sampling Rate    = %.2f MHz\\n', ...  
    Fs_in/1e6);
```

```
fprintf('Output Sampling Rate    = %.2f MHz\\n', ...  
    Fs_out/1e6);
```

```
fprintf('Decimation Factor      = %d\\n', D);
```

```
fprintf('\nBANDWIDTH:\\n');
```

```
fprintf('Original OFDM Bandwidth = %.2f MHz\\n', ...  
    BW/1e6);
```

```
fprintf('Output Nyquist Frequency = %.2f MHz\\n', ...  
    Fnyq_out/1e6);
```

```
fprintf('\nSIGNAL QUALITY:\\n');
```

```
fprintf('Input SNR                = %.2f dB\\n', ...
```

```

    SNR_input_dB);

fprintf('Output SNR          = %.2f dB\n', ...
    SNR_output_dB);

fprintf('EVM                = %.2f %%\n', ...
    EVM_percent);

fprintf('EVM                = %.2f dB\n', ...
    EVM_dB);

fprintf('\nPROCESSING EFFICIENCY:\n');
fprintf('Input Samples      = %d\n', ...
    inputSamples);

fprintf('Output Samples      = %d\n', ...
    outputSamples);

fprintf('Sample Reduction    = %.2f %%\n', ...
    sampleReduction);

fprintf('Sampling Rate Reduction = %.2f %%\n', ...
    samplingRateReduction);

fprintf('Processing Reduction  = %.0f : 1\n', ...
    processingReductionFactor);

fprintf('\n===== \n');

%%
=====

===
% 13. FIGURE 1 - RECEIVED SIGNAL
%
=====

=====

```

```

figure;

numPlot = min(1000,length(rxSignal));

plot((0:numPlot-1)/Fs_in*1e6, ...
     real(rxSignal(1:numPlot)), ...
     'LineWidth',1.2);

grid on;

xlabel('Time (\mus)');
ylabel('Amplitude');

title('Received 5G OFDM Signal');

%%
=====

===
% 14. FIGURE 2 - INPUT SPECTRUM
%
=====

=====

figure;

plot(f_input/1e6, ...
     10*log10(PSD_input + eps), ...
     'LineWidth',1.2);

grid on;

xlabel('Frequency (MHz)');
ylabel('Power/Frequency (dB/Hz)');

title('Input Spectrum');

xlim([-122.88 122.88]);

```

```
%%
```

```
=====
```

```
====
```

```
% 15. FIGURE 3 - FIR FILTER RESPONSE
```

```
%
```

```
=====
```

```
=====
```

```
figure;
```

```
freqz(b,1,4096,Fs_in);
```

```
title('Anti-Aliasing FIR Filter Response');
```

```
%%
```

```
=====
```

```
====
```

```
% 16. FIGURE 4 - FILTERED SPECTRUM
```

```
%
```

```
=====
```

```
=====
```

```
figure;
```

```
plot(f_filtered/1e6, ...  
     10*log10(PSD_filtered + eps), ...  
     'LineWidth',1.2);
```

```
grid on;
```

```
xlabel('Frequency (MHz)');
```

```
ylabel('Power/Frequency (dB/Hz)');
```

```
title('Spectrum After Anti-Aliasing Filtering');
```

```
xlim([-50 50]);
```

```
%%
```

```
=====
```

```
=====
```

```
% 17. FIGURE 5 - DECIMATED OUTPUT SPECTRUM
```

```
%
```

```
=====
```

```
=====
```

```
figure;
```

```
plot(f_output/1e6, ...  
      10*log10(PSD_output + eps), ...  
      'LineWidth',1.2);
```

```
grid on;
```

```
xlabel('Frequency (MHz)');  
ylabel('Power/Frequency (dB/Hz)');
```

```
title('Output Spectrum After Decimation by 8');
```

```
xlim([-15.36 15.36]);
```

```
%%
```

```
=====
```

```
=====
```

```
% 18. FIGURE 6 - INPUT VS OUTPUT
```

```
%
```

```
=====
```

```
=====
```

```
figure;
```

```
plot(f_input/1e6, ...  
      10*log10(PSD_input + eps), ...  
      'LineWidth',1.1);
```

```

hold on;

plot(f_output/1e6, ...
     10*log10(PSD_output + eps), ...
     'LineWidth',1.3);

grid on;

xlabel('Frequency (MHz)');
ylabel('Power/Frequency (dB/Hz)');

title('Input vs Decimated Output Spectrum');

legend('Input Spectrum', ...
       'Output Spectrum');

xlim([-30 30]);

%%
=====

===
% 19. FIGURE 7 - SAMPLING RATE REDUCTION
%
=====

=====

figure;

bar([Fs_in/1e6 Fs_out/1e6]);

set(gca,'XTick',1:2);
set(gca,'XTickLabel',{'Input','Output'});

ylabel('Sampling Rate (MHz)');

title('Sampling Rate Reduction');

```

```
grid on;
```

```
%%
```

```
=====
```

```
====
```

```
% 20. FIGURE 8 - SNR COMPARISON
```

```
%
```

```
=====
```

```
====
```

```
figure;
```

```
bar([SNR_input_dB SNR_output_dB]);
```

```
set(gca,'XTick',1:2);
```

```
set(gca,'XTickLabel',{'Input SNR','Output SNR'});
```

```
ylabel('SNR (dB)');
```

```
title('Signal Quality Comparison');
```

```
grid on;
```

```
%%
```

```
=====
```

```
====
```

```
% 21. IMPORTANT DSP ANALYSIS
```

```
%
```

```
=====
```

```
====
```

```
fprintf('\n=====\\n');
```

```
fprintf('      DSP ANALYSIS\\n');
```

```
fprintf('=====\\n');
```

```
if BW > Fnyq_out
```

```

fprintf('\nBandwidth Limitation:\n');

fprintf(['The original OFDM bandwidth is %.2f MHz, ' ...
        'while the output Nyquist frequency is %.2f MHz.\n'], ...
        BW/1e6, Fnyq_out/1e6);

fprintf(['Therefore, the complete 100 MHz bandwidth ' ...
        'cannot be preserved after decimation by 8.\n']);

fprintf(['The anti-aliasing filter removes frequencies ' ...
        'that cannot be represented at the lower rate.\n']);

end

fprintf('\nProcessing Efficiency:\n');

fprintf(['Sampling rate is reduced from %.2f MHz ' ...
        'to %.2f MHz.\n'], ...
        Fs_in/1e6, Fs_out/1e6);

fprintf(['This provides an %.0fx reduction in sampling rate.\n'], ...
        processingReductionFactor);

fprintf(['Approximately %.2f%% fewer samples need to be ' ...
        'processed downstream.\n'], ...
        sampleReduction);

fprintf('\nConclusion:\n');

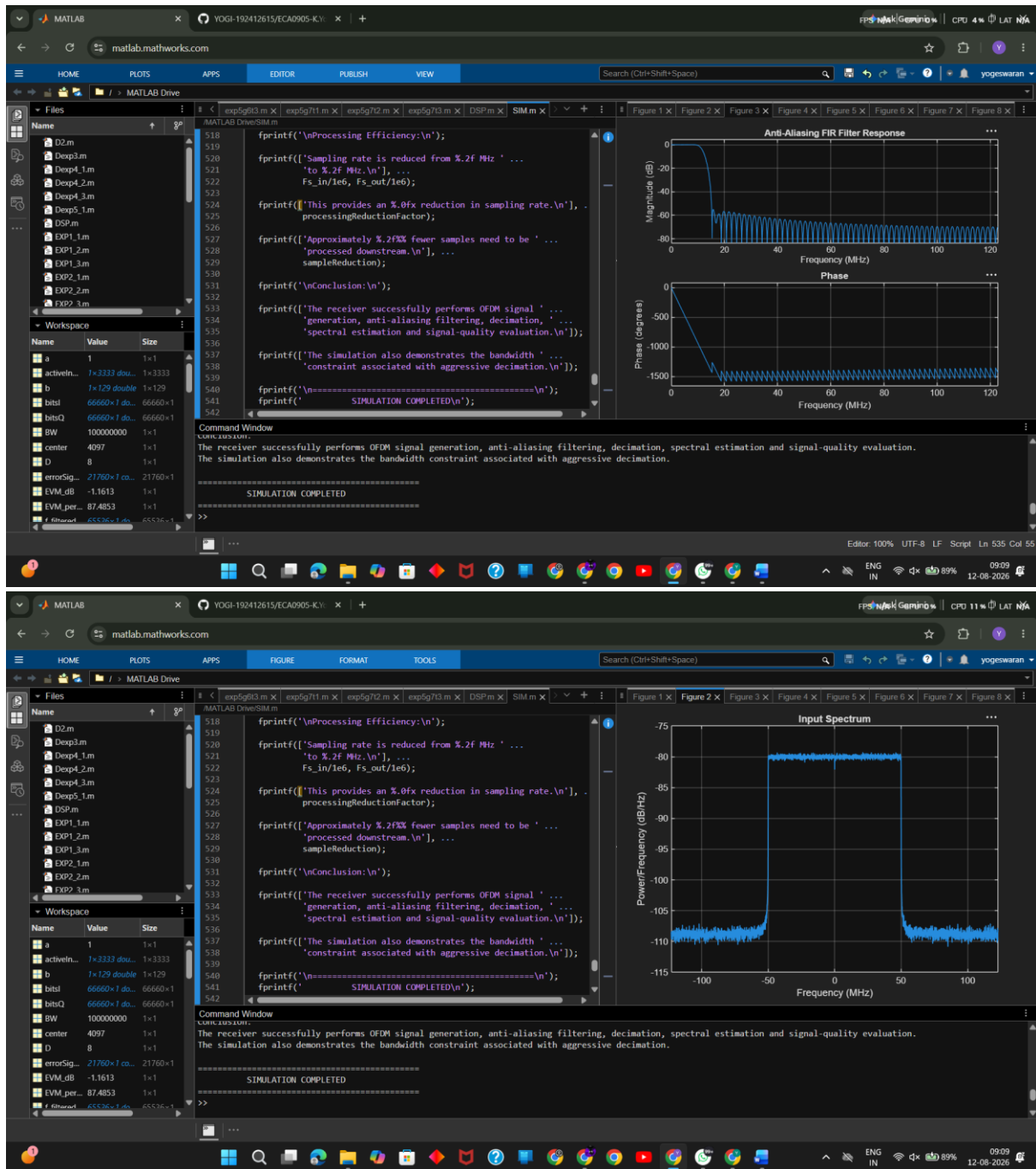
fprintf(['The receiver successfully performs OFDM signal ' ...
        'generation, anti-aliasing filtering, decimation, ' ...
        'spectral estimation and signal-quality evaluation.\n']);

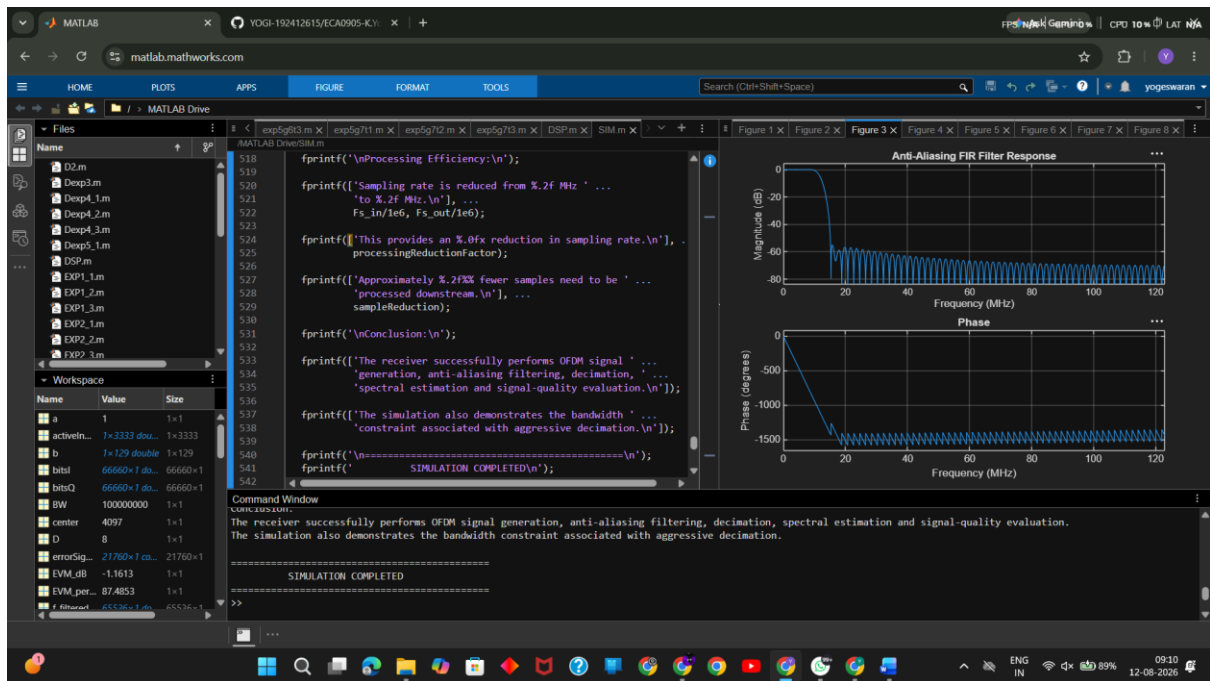
fprintf(['The simulation also demonstrates the bandwidth ' ...
        'constraint associated with aggressive decimation.\n']);

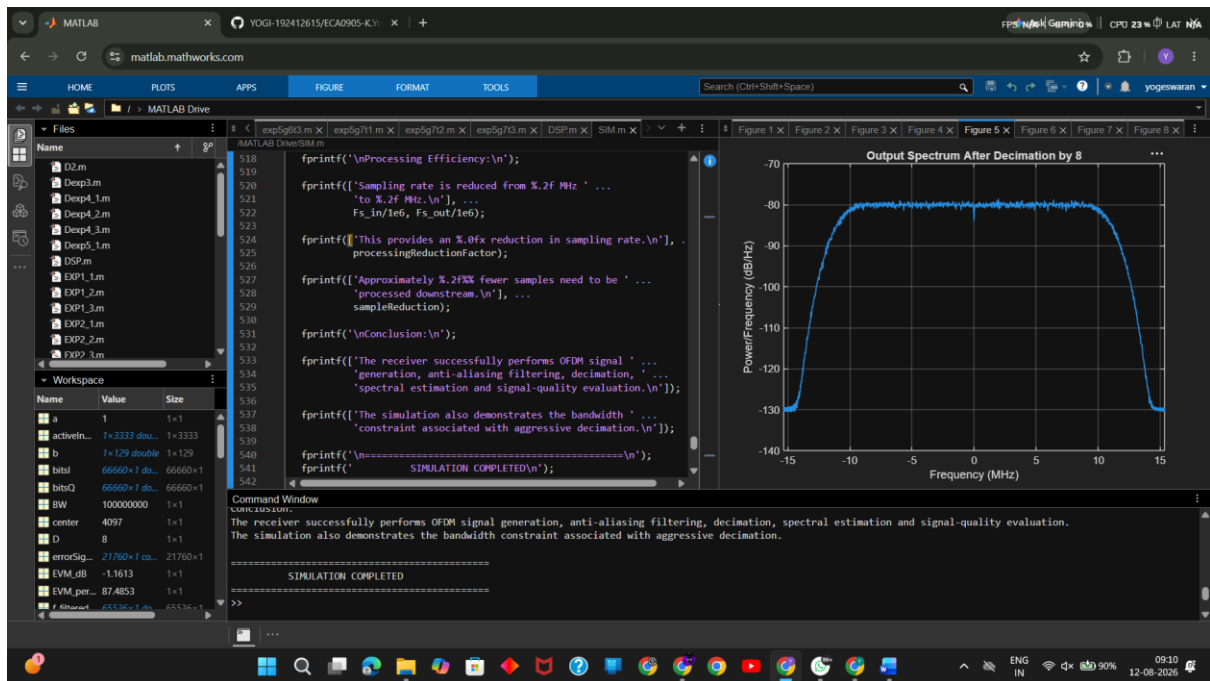
```

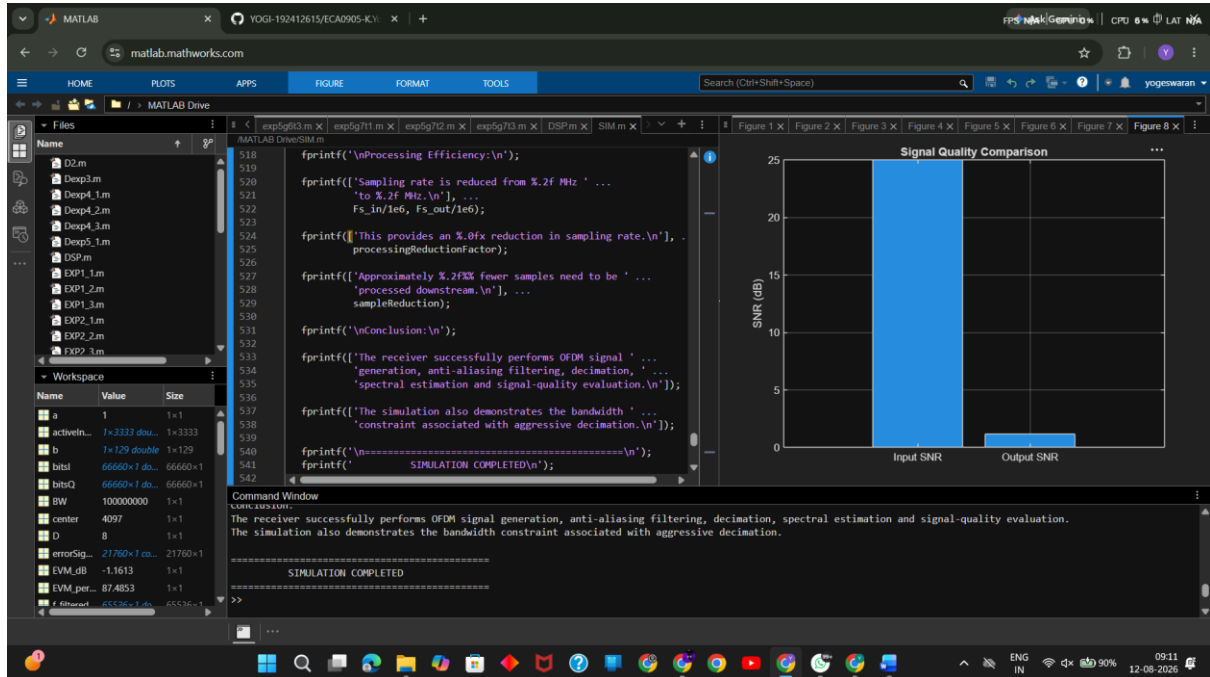
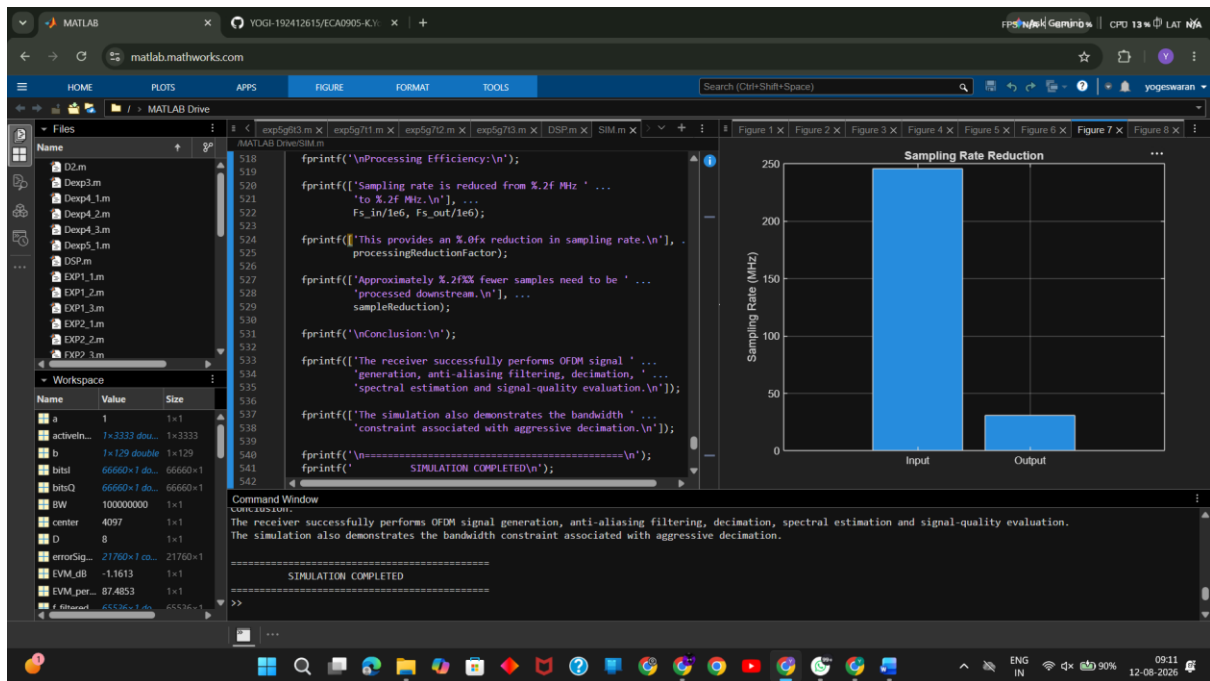
```
fprintf('\n=====\\n');
fprintf('      SIMULATION COMPLETED\\n');
fprintf('=====\\n');
```

OUTPUT









Conclusion

The end-to-end 5G multirate receiver was successfully modeled and simulated using MATLAB. A QPSK-based OFDM signal was generated and passed through an AWGN channel to represent a received communication signal. An anti-aliasing FIR filter was then applied before decimation to prevent aliasing.

The sampling rate was reduced from **245.76 MHz to 30.72 MHz using a decimation factor of 8**, resulting in an **87.5% reduction in sampling rate** and an $8\times$ reduction in the sampling rate required for downstream processing.

Welch's method was used for non-parametric spectral estimation, while SNR and EVM were used to evaluate signal quality.

The simulation also demonstrated an important design constraint: the original **100-MHz OFDM bandwidth cannot be completely preserved after decimation by 8**, because the resulting Nyquist frequency is only **15.36 MHz**. Therefore, appropriate anti-aliasing filtering or an alternative sampling-rate conversion architecture is required.

Overall, the simulation demonstrates the fundamental trade-off between **signal bandwidth preservation and computational efficiency** in a 5G multirate receiver.